



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería en Software

SecureDataMining DevSecOps

Clasificación de Vulnerabilidades mediante Minería de Datos y
Pipelines DevSecOps

Asignatura: Desarrollo de Software Seguro
Docente: Ing. Geovanny Cudco
Integrantes: Cesar Loor
Gabriel Murillo
Camilo Orrico
Fecha: 18 de junio de 2026

Sangolquí, Ecuador

Índice

1. Introducción	2
2. Ingeniería de Datos y SEMMA	2
2.1. Fase SAMPLE: Consolidación de Datasets	2
2.2. Fase EXPLORE: El desafío del Desbalance	2
2.3. Fase MODIFY: Extracción de Características Avanzada	2
3. Modelado y Evaluación (Fase MODEL/ASSESS)	3
3.1. Justificación Algorítmica y Estrategia de Pesos	3
3.2. Explicabilidad mediante SHAP	3
3.3. Desempeño del Modelo	3
4. Análisis Estático Heurístico (SAST)	3
5. Ecosistema DevSecOps: Orquestación e Infraestructura	3
5.1. Contenerización y Despliegue	4
6. Conclusiones	4
7. Bibliografía	4

1 Introducción

El paradigma *Shift-Left Security* propone integrar controles de seguridad lo más temprano posible en el ciclo de desarrollo. Este proyecto materializa dicha visión mediante la creación de un ecosistema autónomo que utiliza **Minería de Datos Analítica** para predecir si un cambio en el código fuente (Pull Request) introduce riesgos de seguridad. La solución orquestada en GitHub Actions combina la velocidad del análisis estático con la profundidad del Machine Learning clásico, eliminando la dependencia de servicios externos costosos.

2 Ingeniería de Datos y SEMMA

2.1 Fase SAMPLE: Consolidación de Datasets

Se diseñó un repositorio de datos unificado que ingesta información de fuentes heterogéneas. La diversidad de lenguajes y contextos de vulnerabilidad es clave para la capacidad de generalización del modelo.

- **CodeXGLUE:** Provee fragmentos de código con anotaciones de defectos semánticos.
- **CVEFixes:** Extrae pares de código "vulnerable/corregido" directamente de repositorios vinculados a la base de datos nacional de vulnerabilidades (NVD).
- **D2A/MegaVul:** Aportan cientos de miles de funciones en C/C++ etiquetadas mediante análisis estático industrial.
- **Total de registros limpios:** 389,313.

2.2 Fase EXPLORE: El desafío del Desbalance

Durante la exploración, se determinó que solo el **8.1%** de los fragmentos de código son vulnerables. Esta asimetría es natural en el desarrollo de software (el código seguro es la norma), pero requiere estrategias de entrenamiento específicas como el *cost-sensitive learning* para evitar que el modelo ignore sistemáticamente la clase minoritaria.

2.3 Fase MODIFY: Extracción de Características Avanzada

La transformación de código fuente a vectores numéricos se realizó mediante tres extractores concurrentes, diseñados para capturar tanto la semántica léxica como la estructura sintáctica:

1. **HashingVectorizer (Truco del Hashing):** Convierte fragmentos de código en vectores de alta dimensionalidad (2^{18}). A diferencia de un vocabulario estático que crece indefinidamente con cada nuevo nombre de variable, el hashing mapea tokens a un espacio fijo, permitiendo procesar cualquier base de código sin necesidad de pre-entrenar un diccionario gigante.
2. **Análisis Estructural AST (Abstract Syntax Tree):** Mediante la integración de la librería industrial **tree-sitter**, el sistema descompone el código en su gramática formal. Se implementó un extractor robusto que identifica:
 - **Pointer Declarators:** Conteo de nodos `pointer_declarator` y `reference_declarator`, fundamentales para identificar riesgos de gestión de memoria en C/C++.
 - **Complejidad Estructural:** Profundidad máxima del árbol y conteo de nodos totales, que sirven como proxies de la mantenibilidad y probabilidad de errores ocultos.
 - **Estructuras de Control:** Frecuencia de `if_statement`, `for_statement` y `method_invocation` para modelar el flujo de ejecución.
3. **Extractor de Taint (Sources/Sinks):** Identifica mediante expresiones regulares optimizadas la presencia de funciones de entrada (*Sources*) como `scanf`, `argv`, `getenv` y funciones de destino peligrosas (*Sinks*) como `strcpy`, `system`, `memcpy`. La coexistencia de ambos en un mismo fragmento eleva drásticamente el peso de la predicción de vulnerabilidad.

3 Modelado y Evaluación (Fase MODEL/ASSESS)

3.1 Justificación Algorítmica y Estrategia de Pesos

Se seleccionó **XGBoost** (Extreme Gradient Boosting) como clasificador principal por su excepcional desempeño en datos tabulares y su capacidad nativa para manejar el desbalance de clases mediante el parámetro `scale_pos_weight`.

La estrategia se basó en el **Cost-Sensitive Learning**: al asignar un peso mayor a la clase minoritaria (11,4), el algoritmo penaliza más severamente el falso negativo (dejar pasar una vulnerabilidad) que el falso positivo. Esto es crítico en seguridad, donde el costo de una brecha es infinitamente superior al de una revisión manual adicional.

3.2 Explicabilidad mediante SHAP

Para superar la naturaleza de "caja negra" de los modelos de boosting, se integró un motor de explicabilidad basado en **SHAP (SHapley Additive exPlanations)**. Esto permite descomponer cada predicción en la contribución individual de cada característica. En el pipeline DevSecOps, esto se traduce en mensajes de retroalimentación útiles para el desarrollador, indicando exactamente qué patrones de código dispararon la alerta de seguridad.

3.3 Desempeño del Modelo

Tabla 1: Métricas detalladas de clasificación

Métrica Global	Valor	Métrica Clase Vulnerable	Valor
Accuracy	92.05 %	Precision	47.69 %
ROC AUC	84.44 %	Recall	30.48 %
CV F1 Mean	0.8122	F1-Score	37.19 %

4 Análisis Estático Heurístico (SAST)

Como red de seguridad adicional, el sistema ejecuta un motor de reglas basado en **CWE (Common Weakness Enumeration)** que garantiza la detección de patrones conocidos independientemente de la probabilidad del modelo ML.

Tabla 2: Reglas de Detección Heurística Implementadas

CWE	Patrón Detectado	Recomendación Técnica
CWE-120	strcpy, strcat, gets	Usar versiones delimitadas (strncpy).
CWE-78	system, popen, exec	Evitar shell indirectos; usar listas de argumentos.
CWE-89	SQL Concat / Template Literals	Implementar <i>Prepared Statements</i> / ORM.
CWE-79	innerHTML, document.write	Sanitizar con DOMPurify o usar textContent.
CWE-134	printf(variable)	Especificar formato estático (printf("%s", var)).
CWE-502	ObjectInputStream.readObject	Migrar a formatos seguros como JSON/Protobuf.

5 Ecosistema DevSecOps: Orquestación e Infraestructura

El pipeline de seguridad implementado en **GitHub Actions** actúa como el sistema nervioso del proyecto, orquestando múltiples etapas de validación:

1. **Status Checks y Reglas de Protección:** El resultado de la inferencia de IA se reporta directamente a la API de GitHub como un chequeo de estado obligatorio. Si el modelo identifica un riesgo alto, el botón de "Merge" se bloquea, forzando una revisión manual.

2. **Gestión Automática de Vulnerabilidades:** Ante una detección positiva, el pipeline:

- Crea un **GitHub Issue** con el label `fixing-required`, vinculándolo automáticamente al Pull Request original.
- Publica un comentario detallado en el PR con recomendaciones específicas por CWE y los resultados de la explicabilidad del modelo.

3. **Notificaciones Multi-Canal (Telegram):** Se implementó un bot de Telegram que consume la API de mensajes para notificar instantáneamente al equipo sobre el estado de cada etapa (Inferencia, Pruebas Unitarias y Despliegue en Render).

5.1 Contenerización y Despliegue

Para asegurar la reproducibilidad del entorno de ejecución, se diseñó un **Dockerfile** multietapa que empaqueta todas las dependencias (Python, Tree-Sitter, XGBoost). Esto permite que la API de inferencia pueda desplegarse de forma elástica en plataformas como **Render**, garantizando que el servicio de escaneo esté siempre disponible y aislado de conflictos de dependencias locales.

6 Conclusiones

La solución implementada demuestra un equilibrio entre rigor analítico y operatividad DevSecOps. El uso de la metodología SEMMA permitió pasar de una recolección de datos masiva a un sistema de decisión distribuido que educa al desarrollador en lugar de solo castigar el error. La integración de la ingeniería de características (AST + Hashing) dota al modelo de una visión semántica del código que supera el análisis de texto simple.

7 Bibliografía

Referencias

- [1] OWASP Top 10:2025, “The Most Critical Web Software Security Risks.”
- [2] Chen, T., & Guestrin, C., “XGBoost: A Scalable Tree Boosting System,” 2016.
- [3] SAS Institute, “SEMMA Methodology Overview,” 2026.